

REDES NEURAIS ARTIFICIAIS E MACHINE LEARNING

TEMA 1 – PERCEPTRON DE MÚLTIPLAS CAMADAS

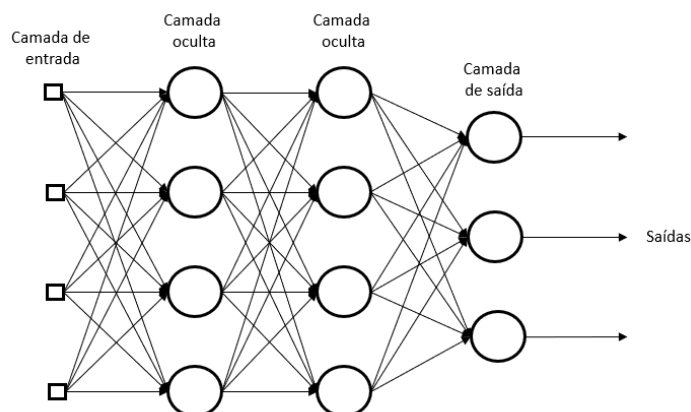
No estudo do perceptron de camada única, é possível perceber o potencial desse tipo de RNA para tarefas de classificação. O algoritmo de aprendizagem LMS, em particular, fornece um método poderoso de descida de gradiente para a redução gradativa do erro, ainda que os padrões não sejam linearmente separáveis. Entretanto, a classe de soluções que podem ser obtidas a partir dessas redes, em cujo espaço de pesos há uma divisão por meio de um hiperplano (também chamado de **discriminante linear**), embora surpreendentemente boa em um conjunto de problemas do mundo real, não é suficiente em aplicações de maior exigência. Há muitos problemas em relação aos quais discriminantes lineares são insuficientes para que a RNA possa alcançar o mínimo erro (Duda; Hart; Stork, 2001).

O **perceptron de múltiplas camadas** (Multilayer Perceptron – MLP) surge como uma alternativa que pode ser generalizada para uma ampla gama de problemas de classificação. A adoção de uma RNA que contenha camadas ocultas, além das camadas de entrada e saída, amplia consideravelmente o número de pesos, o que por sua vez permite uma aprendizagem capaz de contemplar também a classificação em conjuntos de forma não linear.

Estendendo o conceito de aprendizagem pelo algoritmo LMS, que se adequa às redes, tais como o perceptron de camada única ou o adaline, o MLP contém uma ou mais camadas ocultas, pelas quais o sinal de entrada relativo aos conjuntos de amostras da RNA se propaga, através de neurônios presentes nessas camadas ocultas, transformando o sinal por meio de funções de ativação, até que o sinal chegue à camada de saída, o que permite que a rede classifique tal conjunto de amostras (Figura 1).

Da mesma forma que o algoritmo LMS faz a retroação de parte do sinal de saída, ajustando os valores dos pesos sinápticos em direção ao menor erro, no MLP o **algoritmo de retropropagação de erro** (*error backpropagation*) possibilita que uma parte do sinal de saída da rede seja alimentada em sentido contrário às camadas (Haykin, 2001, p. 183). O algoritmo de retropropagação implementa dois tipos de retroalimentação: i) da camada de saída à camada oculta posicionada anteriormente a ela; e ii) da camada oculta para a camada oculta, até que se alcance a primeira camada com pesos sinápticos.

Figura 1 – Exemplo de um perceptron de múltiplas camadas



Fonte: Elaborado com base em Haykin, 2001, p. 186.

No exemplo da Figura 1, podemos notar a quantidade de pesos sinápticos que surgem devido à presença de duas camadas ocultas. Contendo somente a camada de entrada e saída, o número de pesos sinápticos é 12 (4 entradas por 3 saídas). Com as duas camadas ocultas, o número de pesos aumenta consideravelmente, para 44 (4 entradas por 4 na primeira camada oculta – 16; mais 4 da primeira para 4 da segunda camada oculta – 16; mais 4 da camada oculta para 3 da camada de saída – 12). No entanto, a simples presença de mais camadas ocultas não significa necessariamente melhoria de forma contínua na aprendizagem de um MLP.

As três características distintivas de um MLP sobre outras RNA são destacadas a seguir (Haykin, 2001, p. 184):

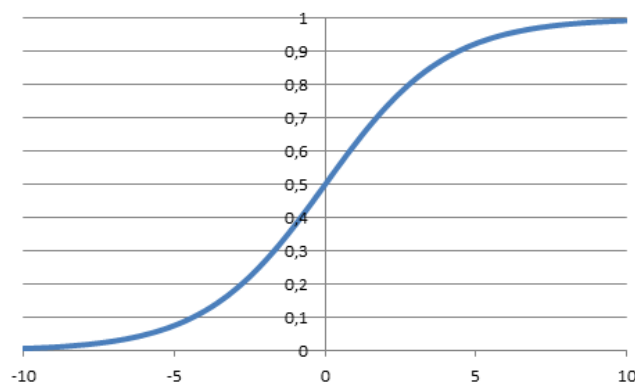
1. Uso de **função de ativação não linear**: a diferença com a forma de ativação vista no modelo do perceptron de camada única é a mudança **suave** proporcionada pela ativação dos neurônios. A função **sinal** no algoritmo LMS causava uma mudança abrupta na ativação. Uma das funções mais utilizadas para se conseguir este efeito no MLP é a função logística ou função sigmóide (figura 2):

$$y_i = \frac{1}{1 + \exp(-v_j)}$$

2. Existência de **camadas ocultas**, que não fazem parte nem da entrada e nem da saída.

3. Alto grau de conectividade do MLP, em função de sua arquitetura e da população de pesos sinápticos.

Figura 2 – Gráfico da função logística ou função sigmoide



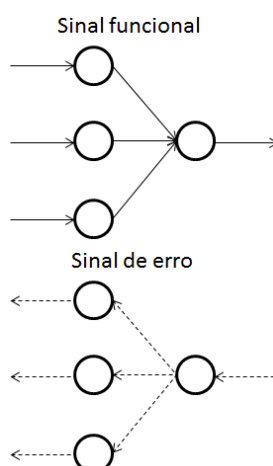
Fonte: Elaborado com base em Medeiros, 2018, p. 153.

A propagação de sinais do MLP envolve o processamento de dois tipos de sinais (Figura 3), que se propagam pela rede (Haykin, 2004, p. 186-187):

- **Sinal funcional:** é o sinal apresentado à camada de entrada referente aos atributos do vetor de amostras, que se propaga para a frente na rede, nó por nó, ativando os neurônios até a camada de saída.
- **Sinal de erro:** tem origem em um neurônio da camada de saída, porém se propaga para trás na rede, ajustando os valores dos pesos ou sinapses.

A Figura 3 apresenta o sinal funcional, que se propaga à frente (*feed-forward*), e o sinal de erro (*error signal*), que retropropaga na rede (*feedback*).

Figura 3 – Representação do sinal funcional e do sinal de erro



Fonte: Elaborado com base em Haykin, 2001, p. 186.

O algoritmo de retropropagação para o MLP envolve o processo chamado de **descida de gradiente**, que visa apresentar o cálculo do **gradiente local do erro** (direção para onde tende a crescer o valor do erro médio calculado), utilizando-o para corrigir os pesos sinápticos na direção contrária ao gradiente, em busca do erro mínimo local.

Quadro 1 – Notação utilizada para o detalhamento do algoritmo de retropropagação

Variável	Descrição
i, j, k	Índices dos neurônios. A partir da notação em que o sinal funcional se propaga da esquerda para a direita, o neurônio i está numa camada mais à esquerda; o índice j nas camadas ocultas; e o índice k na camada de saída.
E	Erro global; soma dos erros quadráticos.
e_j	Sinal de erro na saída do neurônio j .
d_j	Sinal desejado no neurônio de saída j .
y_j	Sinal funcional que aparece na saída do neurônio j .
x_j	Sinal da amostra apresentada no neurônio j da camada de entrada.
w_{ji}	Peso sináptico conectando a entrada do neurônio i à saída do neurônio j .
Δw_{ji}	Correção aplicada ao peso sináptico que conecta a entrada do neurônio i à saída do neurônio j .
v_j	Soma ponderada de todas as entradas sinápticas no neurônio j acrescida do bias.
δ_j	Gradiente local para o neurônio j .
f	Função de ativação.
η	Taxa de aprendizado.
α	Taxa de momentum.

Fonte: Elaborado com base em Medeiros, 2018, p. 151.

TEMA 2 – ALGORITMO DE RETROPROPAGAÇÃO, CARACTERÍSTICAS DE TREINAMENTO DE UM MLP, GENERALIZAÇÃO E VALIDAÇÃO CRUZADA

Vejamos as etapas do algoritmo de retropropagação para o treinamento de um MLP:

1. Construa uma rede MLP com arquitetura totalmente conectada, com N neurônios na camada de entrada, M neurônios na camada de saída e P

neurônios na camada oculta, adaptados às características do problema em questão.

2. Os pesos sinápticos devem ser inicializados com valores aleatórios e os pesos w_0 (bias) com valor +1.
3. Escolha uma amostra na forma de um par entrada-saída desejada (x_i, d_i) . Na camada de entrada, os valores do neurônio y_i coincidem com os x_i .
4. Faça a propagação do sinal de cada neurônio da camada de entrada y_i para a camada oculta intermediária, efetuando o somatório ponderado pelos pesos sinápticos w_{ji} para calcular v_j (chamado de **campo local induzido**), e utilize a função de ativação para calcular os valores de entrada y_j para a camada intermediária:

$$v_j = \sum_{i=0}^N w_{ji} y_i$$

E:

$$y_j = f(v_j)$$

A função logística ou sigmoide tem um comportamento mais “suave” do que a função sinal utilizada no exemplo do perceptron de camada única. Enquanto a função degrau apresentava apenas duas opções (+1 ou -1), a função sigmoide calcula valores infinitos, dependendo do argumento da função. Assim, enquanto a função sinal é uma função **discreta**, a função sigmoide é considerada uma função **contínua**. A fórmula da função sigmoide¹ é a seguinte:

$$f(v_j) = \frac{1}{1 + e^{-av_j}}, a > 0$$

5. Faça a propagação do sinal dos neurônios da camada intermediária y_j para a camada de saída por meio do somatório ponderado pelos pesos sinápticos w_{kj} , para calcular o campo local induzido v_k , utilizando a função sigmoide para calcular os valores de entrada y_k para a camada de saída:

$$v_k = \sum_{j=0}^P w_{kj} y_j$$

¹ O símbolo e utilizado é o número de Euler (aproximadamente 2,71828...). Não confundir com o valor do erro e_j . A variável a regula a inflexão da curva sigmoide, sendo comum adotar $a=1$.

E:

$$y_k = f(v_k)$$

6. Calcule o gradiente local para os neurônios da camada de saída δ_k de acordo com a fórmula (no caso de se utilizar a função sigmoide)

$$\delta_k = ay_k(d_k - y_k)(1 - y_k)$$

7. Calcule o gradiente local para os neurônios da camada intermediária δ_j , conforme a fórmula:

$$\delta_j = ay_j(1 - y_j) \sum_{k=0}^M \delta_k w_{kj}$$

8. Atualize o valor dos pesos conforme os valores dos gradientes locais calculados para a camada de saída e a camada intermediária. O índice sobrescrito $t+1$ indica o próximo valor das atualizações dos pesos, sendo o índice sobrescrito t o valor atual.

$$\Delta w_{kj}^{t+1} = \alpha \Delta w_{kj}^t + \eta \delta_k y_k$$

E:

$$\Delta w_{ji}^{t+1} = \alpha \Delta w_{ji}^t + \eta \delta_j y_j$$

O valor de atualização compõe-se de duas partes: uma relativa ao valor atual do peso, quando há multiplicação pela **taxa de momento** α ; e outra referente à modificação calculada pelo gradiente local, com multiplicação pela **taxa de aprendizagem** η . Os pesos sinápticos devem ser atualizados:

$$w_{kj}^{t+1} = w_{kj}^t - \Delta w_{kj}^t$$

E:

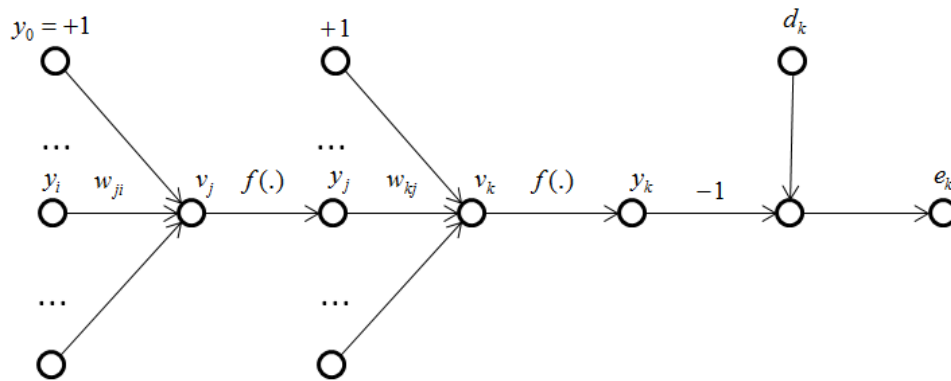
$$w_{ji}^{t+1} = w_{ji}^t - \Delta w_{ji}^t$$

9. Retorne ao passo 3 e escolha outro par entrada-saída. Quando todos os pares forem apresentados à rede, isso significa que uma **época de treinamento** foi alcançada. Deve-se refazer o processo para várias épocas até que o valor global de erro alcance um valor que possibilite a classificação ótima pela rede MLP. O erro global pode ser calculado pela

seguinte fórmula, em que C indica o somatório para todas as amostras apresentadas ao perceptron multicamada:

$$E = \frac{1}{2} \sum_C e_k^2 = \frac{1}{2} \sum_C (d_k - y_k)^2$$

Figura 4 – Representação da propagação do sinal funcional à frente no MLP, da entrada, passando pela ativação até finalizar na camada de saída



Fonte: Elaborado com base em Haykin, 2001, p. 192.

O algoritmo de retropropagação é o mais comumente utilizado com o treinamento de MLP. Vários algoritmos propostos também permitem o treinamento de um MLP, como RProp (Riedmiller; Braun, 1992), QuickProp (Fahlman, 1989), método do gradiente conjugado (Fletcher; Reeves, 1964), algoritmo de Levenberg-Marquardt (Marquardt, 1963), e filtro de Kalman (1960), dentre outros.

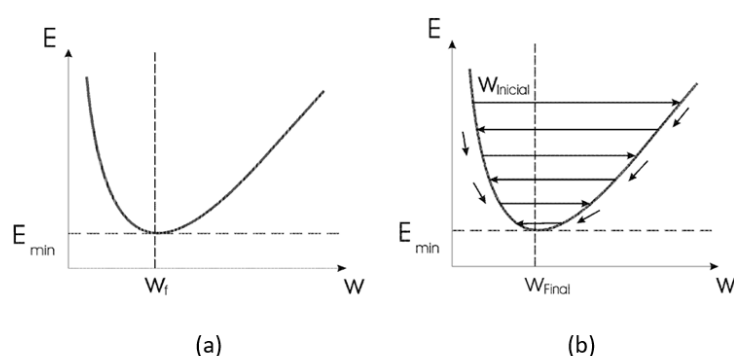
2.1 Parâmetros de Aprendizagem de um MLP

O processo de aprendizagem (ou treinamento) de um MLP busca garantir o menor erro global. Essa busca pode ser ilustrada de maneira visual relacionando graficamente o erro global com os pesos sinápticos. O erro global pode ser representado como uma hipersuperfície, em que o ponto representado pelo valor dos pesos “desliza” (Figuras 5 e 6). O cálculo do gradiente local nas etapas 6 e 7 do algoritmo de retropropagação fornece a direção, a partir da qual encontramos o valor do erro máximo; por isso, a variação Δw_{ji} deve ser negativa, para fazer com que os pesos sinápticos sigam em direção ao ponto de mínimo erro local.

Essa representação da busca do erro mínimo, como um ponto representado pelos pesos sinápticos que “surfam” sobre a superfície de erro,

ajuda a entender também o papel dos parâmetros da **taxa de aprendizagem** e da **taxa de momento**. A taxa de aprendizagem refere-se ao tamanho do vetor contrário ao gradiente: se for maior, pode passar do ponto de mínimo local e ficar circulando em volta do ponto; se for menor, pode fazer com que os pesos se desloquem gradativamente para o ponto de mínimo de forma suave; se for muito menor, pode fazer com que a aprendizagem seja demorada (Figura 7).

Figura 5 – Dimensão de peso sináptico com o erro global

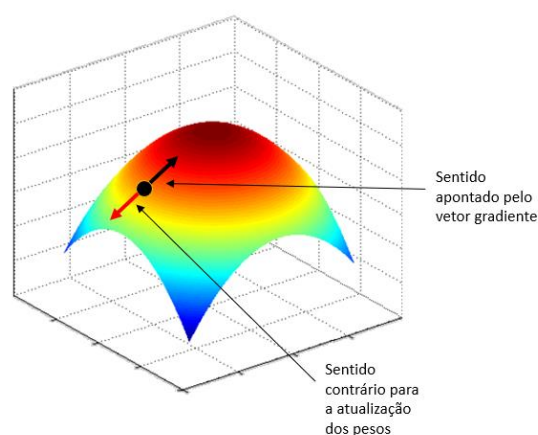


Legenda: (b) o processo de atualização dos pesos é visualizado como a convergência do valor dos pesos a um valor final ótimo.

Fonte: Elaborado com base em de Medeiros, 2018, p. 144.

A taxa de momento, que ajusta a proporção de atualização dos valores atuais dos pesos, refere-se à forma como o ponto dos pesos deve ficar “parado”: se a taxa de aprendizagem for grande, o valor relativo ao momento pode fazer com que os pesos não se movimentem muito rápido. A taxa de momento funciona, portanto, como força de inércia para o processo de aprendizagem, auxiliando a regular a taxa de aprendizagem e suavizando a busca pelo ponto de mínimo local.

Figura 6 – Ponto sobre uma superfície de erro de um espaço de pesos



O cálculo do gradiente local dá o sentido de maior valor do erro; a atualização dos pesos deve ser feita no sentido contrário, indicando onde deve estar localizado o valor mínimo do erro.

Entretanto, em certos problemas, podem surgir condições que geram espaços de busca que contêm mais de um ponto de mínimo. Dizemos que, quando um valor de erro global converge para um valor mínimo, ainda que não seja o mais baixo do espaço de busca, trata-se de um **mínimo local**. Dessa forma, o valor mínimo mais baixo de um espaço de busca é chamado de **mínimo global** (Figura 8). O algoritmo de retropropagação de um MLP pode ficar preso em um ponto de mínimo local, devido à descida gradativa que o algoritmo de treinamento faz nessa curva. Certas modificações nos algoritmos podem ser feitas de modo a reduzir a possibilidade de a aprendizagem ficar presa em um ponto de erro mínimo local.

Figura 7 – Comportamento do aprendizado do MLP para diferentes taxas de aprendizagem

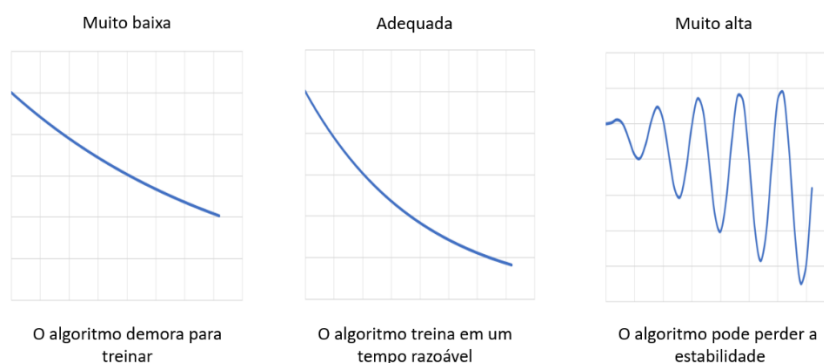
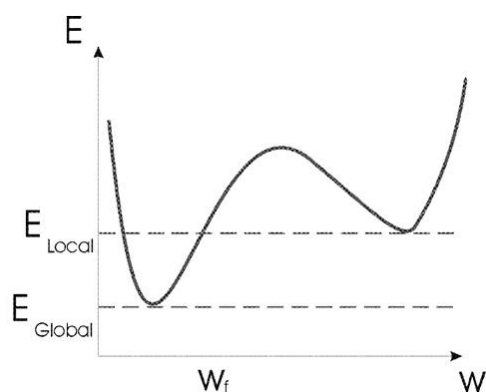


Figura 8 – Representação de espaço de busca: ponto de erro mínimo local e erro mínimo global



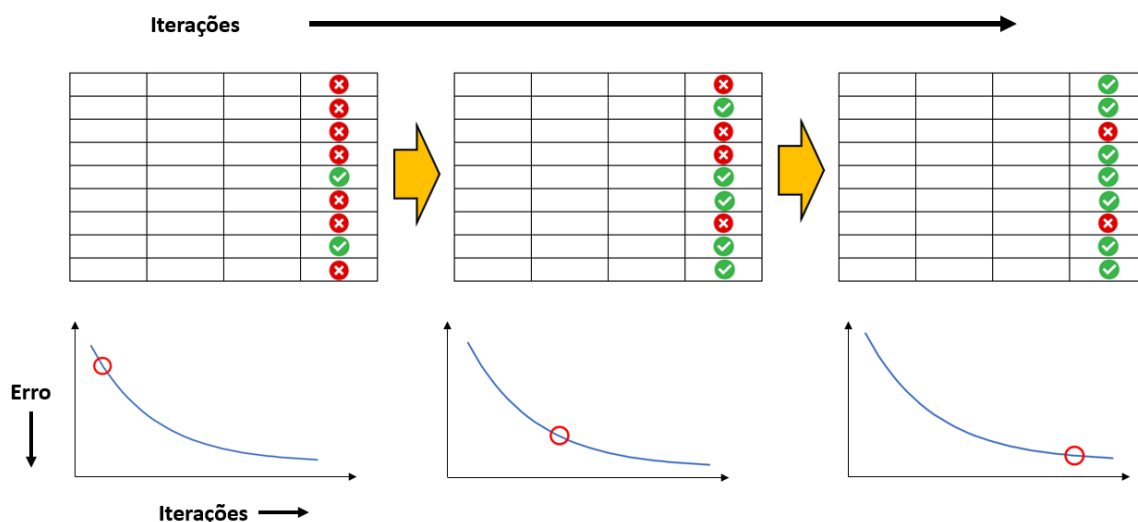
Fonte: Elaborado com base em de Medeiros, 2018, p. 145.

2.2 Treinamento e generalização

O treinamento de um MLP consiste na construção de um modelo que visa diminuir gradativamente o erro de classificação das amostras do conjunto de entrada, também chamadas de **instâncias**. Dessa forma, fica bem caracterizado como um processo de aprendizagem por **tentativa e erro**: o modelo vai ficando mais eficiente, à medida que aprende em qual rótulo classificar uma instância do conjunto de treinamento. O algoritmo de retropropagação será executado um determinado número de vezes, ou **iterações**. Após um critério de parada, que pode ser definido pelo número máximo de iterações ou pelo alcance de um valor de erro mínimo, o algoritmo termina e o MLP apresenta os valores dos pesos que demonstram o seu aprendizado.

Ao longo das iterações, as instâncias são apresentadas. Após a retropropagação do sinal de erro para trás na MLP, os sinais são alimentados para frente novamente, de maneira que as instâncias que eram classificadas de forma errônea começam a ser gradativamente classificadas da forma esperada. Na Figura 9, podemos visualizar esse processo: de acordo com as iterações, o MLP vai executando a tarefa, passando a acertar mais e a errar menos. Assim, curva de aprendizagem tende a mostrar um valor de erro global diminuindo.

Figura 9 – Ilustração do processo de aprendizagem de um MLP

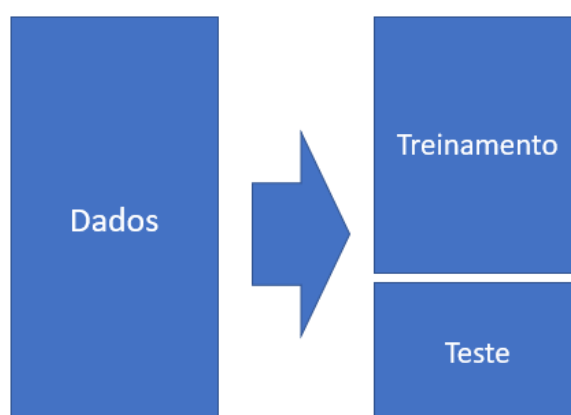


Uma vez que o modelo é gerado, é necessário avaliar o seu desempenho, principalmente com relação a amostras ou instâncias que não fizeram parte do conjunto utilizado para o treinamento. Nessa fase, os dados são conhecidos como dados de **teste** ou dados de **validação**. Em geral, quando se utiliza um conjunto

de dados (ou *dataset*, como normalmente é chamado), é conveniente dividi-lo em duas partes: uma será utilizada na **fase do treinamento** e outra na **fase de teste**. Previamente ao processo de treinamento, deve haver uma etapa de organização do *dataset*, que deve ser dividido para as duas fases. Como o *dataset* tem os rótulos (classes) para cada instância do conjunto de teste, é possível comparar a estimativa dada pelo MLP com a pré-classificação. Dessa forma, o desempenho do modelo preditivo construído pelo MLP fornece uma estimativa de sua capacidade de **generalização**. A generalização é a habilidade do modelo construído de responder corretamente a dados não utilizados no treinamento.

Na Figura 10, vemos a organização do conjunto de instâncias em duas partes: o conjunto de treinamento, utilizado no algoritmo de retropropagação; e o conjunto de teste, que será utilizado para validar o treinamento.

Figura 10 – Organização do conjunto de instâncias



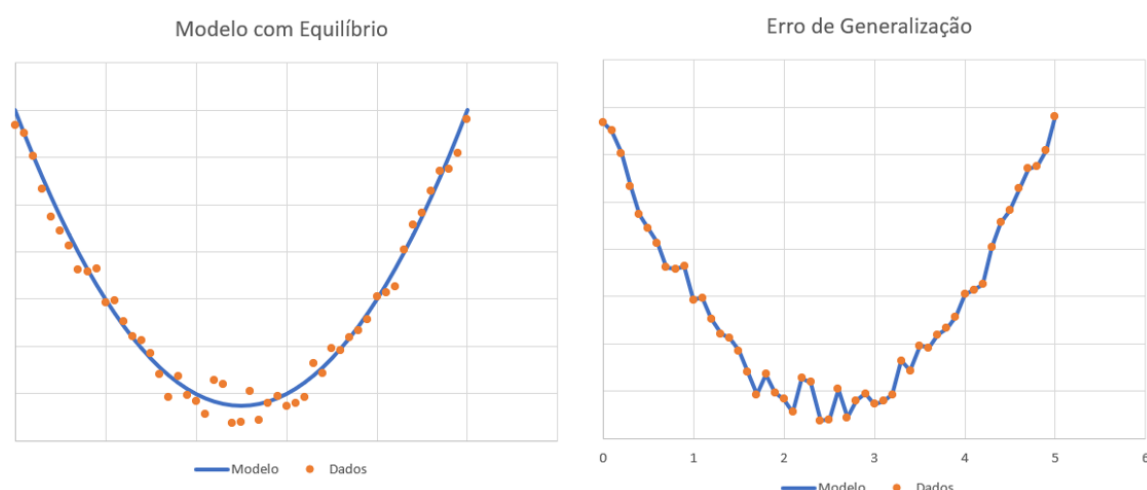
Diz-se que uma rede *generaliza* bem quando o mapeamento entrada-saída computado pela rede for correto (ou aproximadamente correto) para dados de teste não-utilizados para a criação ou treinamento da rede; o termo “generalização” é tomado emprestado da psicologia. Aqui assume-se que os dados de teste são retirados da mesma população usada para gerar os dados de treinamento. (Haykin, 2001, p. 232)

Ainda que o modelo possa, ao final do treinamento, exibir um erro global pequeno, isso não quer dizer que o MLP está bem ajustado aos dados utilizados no conjunto de entrada. Se o modelo não consegue representar bem os dados de origem, pode não ser capaz de fornecer uma boa generalização. Uma aproximação equilibrada indica que o modelo consegue representar bem os dados, com baixo desvio do predito em relação às instâncias (variância). Ainda que existam ruídos, o modelo consegue representar bem o conjunto de dados, ou

seja, o modelo é capaz de **generalizar** bem os dados de entrada. Dizemos ainda que o modelo faz uma boa **interpolação** dos dados.

A Figura 11 ilustra o fenômeno da generalização. À esquerda, uma boa generalização mostra um padrão por trás do modelo alcançado, a partir do qual existe pouca variância dos dados, que apresentam erro mínimo quando comparados ao modelo. À direita, os dados foram treinados em excesso, de maneira que o modelo alcançado só está adequado ao conjunto de treinamento, podendo apresentar alto grau de erro com relação a dados de teste. Ou seja, ele se ajusta praticamente sem erro com os dados de entrada, mas pode não generalizar adequadamente os dados de teste. Nesse caso, o modelo é **sobregeneralizado**.

Figura 11 – Modelos com equilíbrio e erro de generalização



Uma das técnicas utilizadas para alcançar uma boa generalização é a **validação cruzada** (*cross validation*). O conjunto de dados disponível é dividido de maneira aleatória nos dois conjuntos, de treinamento e de teste, conforme a Figura 10. A ideia é efetuar uma série de treinamentos do MLP, de maneira a encontrar o modelo de melhor generalização, que apresente a menor margem de erro em relação aos dados de teste. Ao longo desses treinamentos, os parâmetros livres (pesos sinápticos, bias, número de neurônios nas camadas ocultas e número de camadas ocultas) são modificados visando alcançar o modelo de melhor generalização.

É preciso apontar também que a explicação se concentra nas tarefas de classificação, em que os perceptrons de múltiplas camadas conseguem ter um bom desempenho, servindo como base para exemplos que serão estudados

adiante. Outras tarefas, como regressão e filtragem, também utilizam MLP, mas não serão aprofundados aqui.

Na continuidade do estudo dos perceptron de múltiplas camadas, a ideia é construir um modelo de MLP com base em uma abordagem prática e fazer a sua experimentação. Para isso, será utilizada uma das ferramentas mais populares para a prática de inteligência artificial e ciência de dados: o Jupyter Notebook. Baseado em linguagem Python, a ferramenta permite um alto nível de interatividade e flexibilidade, próprio de atividades relacionadas à modelagem de soluções.

TEMA 3 – PYTHON, JUPYTER NOTEBOOK, BIBLIOTECAS (NUMPY, PANDAS)

A linguagem Python tem se tornado uma referência em aplicações que utilizam Inteligência Artificial e Machine Learning, devido ao alto nível de interatividade e flexibilidade que oferece à construção de modelos. Apesar de ter se popularizado nos últimos anos, a linguagem Python surgiu em 1991. Como linguagem aberta, diferentemente de outras linguagens, como C e Java, ela é **interpretada**: o comando do usuário é executado logo após a digitação no software interpretador, dispensando a necessidade de compilar o código.

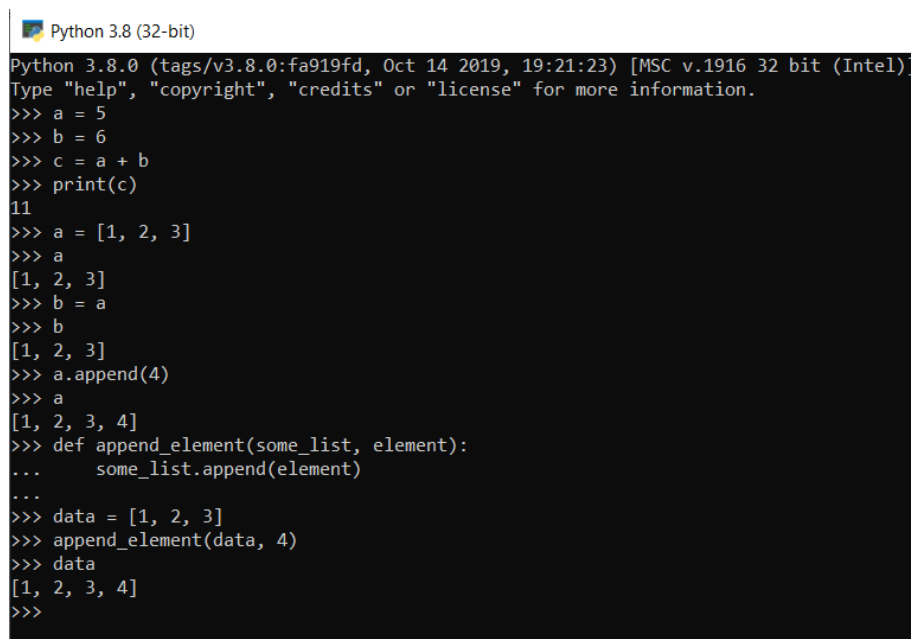
A Python apresenta uma implementação básica de comandos de programação. Mas a linguagem é enriquecida por um grande número de bibliotecas ou API, criadas por desenvolvedores que formam uma comunidade de processamento científico e análise de dados. Dessa forma, além das bibliotecas padrão, outras bibliotecas podem ser instaladas, de acordo com a necessidade de processamento do usuário (Müller; Guido, 2015).

A linguagem Python implementa estruturas de dados de alto nível, orientadas para o processamento eficiente dos dados. Apresenta ainda uma implementação simples e eficaz de programação orientada a objetos. Com uma sintaxe elegante, com poucos símbolos, a linguagem é ideal para a execução de códigos pequenos e rápidos, buscando atingir objetivos específicos (*scripts*). A sintaxe da linguagem Python foi se modificando ao longo de suas versões. A versão atual é a 3.X. Ainda que seja possível desenvolver scripts na versão 2.X, tal procedimento não é recomendável, pela falta de retrocompatibilidade.

O interpretador Python permite a execução do *shell*, através do qual é possível inserir comandos (Figura 12). Os comandos podem ser fornecidos na própria tela, e o resultado será apresentado logo em seguida. Entretanto, para

tarefas mais complexas de análise, é interessante editar os comandos em um arquivo com extensão `.py` (Kopec, 2019).

Figura 12 – Shell do Python 3.8



```
Python 3.8 (32-bit)
Python 3.8.0 (tags/v3.8.0:fa919fd, Oct 14 2019, 19:21:23) [MSC v.1916 32 bit (Intel)]
Type "help", "copyright", "credits" or "license" for more information.
>>> a = 5
>>> b = 6
>>> c = a + b
>>> print(c)
11
>>> a = [1, 2, 3]
>>> a
[1, 2, 3]
>>> b = a
>>> b
[1, 2, 3]
>>> a.append(4)
>>> a
[1, 2, 3, 4]
>>> def append_element(some_list, element):
...     some_list.append(element)
...
>>> data = [1, 2, 3]
>>> append_element(data, 4)
>>> data
[1, 2, 3, 4]
>>>
```

Para uma abordagem em alto nível, a linguagem Python também pode ser utilizada com base em IDEs desenvolvidas para aumentar a produtividade. Algumas das IDEs mais utilizadas são PyCharm, PyDev, Python Tools para Visual Studio, Spyder (Anaconda) e Komodo. Algumas IDEs permitem a instalação de plugins que permitem a customização das plataformas.

O poder da linguagem Python provém em grande parte da instalação de bibliotecas para o processamento eficiente de dados e para a execução de diversas tarefas. Vejamos algumas bibliotecas mandatórias para a maioria dos processos relacionados com ciência de dados e inteligência artificial:

- **Numpy**: nome proveniente da abreviatura de *Numerical Python*. É a pedra angular do processamento numérico em Python. Oferece funções para efetuar processamentos em todos os elementos de *arrays* ou operações matemáticas entre *arrays*. Apresenta ainda ferramentas para ler e gravar conjuntos de dados baseados em arrays de disco. A biblioteca Numpy tira proveito do hardware instalado (GPU, processamento paralelo etc.) para maximizar a sua performance.
- **Pandas**: de forma complementar ao Numpy, a biblioteca Pandas oferece estruturas de dados de alto nível, com funções projetadas para dados

estruturados ou tabulares, de forma semelhante a tabelas de bancos de dados. Duas estruturas de dados muito utilizadas da biblioteca Pandas são o *DataFrame* (estrutura tabular orientada a colunas, com rótulos tanto para linhas quanto para colunas) e o *Series* (objeto array unidimensional com rótulo).

- **Matplotlib:** uma das bibliotecas mais populares para a construção de gráficos e plotagens, gera ainda diversas visualizações de dados bidimensionais. Foi projetada para a criação de plotagens apropriadas para a publicação de dados. Permite integração direta com formatos de dados de outras bibliotecas, para a visualização dos dados.
- **Scipy:** trata-se de uma coleção de pacotes voltada para uma série de diversos domínios de problemas que são padrão em processamento científico. Apresenta um ecossistema de software *open source* para matemática, ciências e engenharia.
- **Scikit-learn:** é a principal “caixa” de ferramentas de propósito geral para aprendizado de máquina dos programadores Python. Ela inclui vários submódulos próprios para as tarefas de preparação de dados em ciência de dados e inteligência artificial: classificação, regressão, agrupamento, pré-processamento, redução de dimensionalidade e seleção de modelos.
- **Statsmodels:** consiste em um pacote de análise estatística, com algoritmos para estatística clássica, modelos de regressão, análise de variância, séries temporais e visualização de dados. Foco maior em inferência estatística.

Uma das ferramentas mais práticas para a execução de scripts em Python é **Jupyter Notebook**. Trata-se de um aplicativo da web de código aberto, que permite criar e compartilhar *notebooks*: documentos que contêm código ativo, equações, visualizações e texto narrativo. Seu uso inclui: limpeza e transformação de dados, simulação numérica, modelagem estatística, visualização de dados, aprendizado de máquina e mais. Como plataforma de execução para Python, grandes empresas oferecem ambientes para o desenvolvimento no formato *notebook*, como o Google Colaboratory². Podem ser instalados pelo usuário em ambiente local, utilizando ferramentas como o Anaconda Navigator.

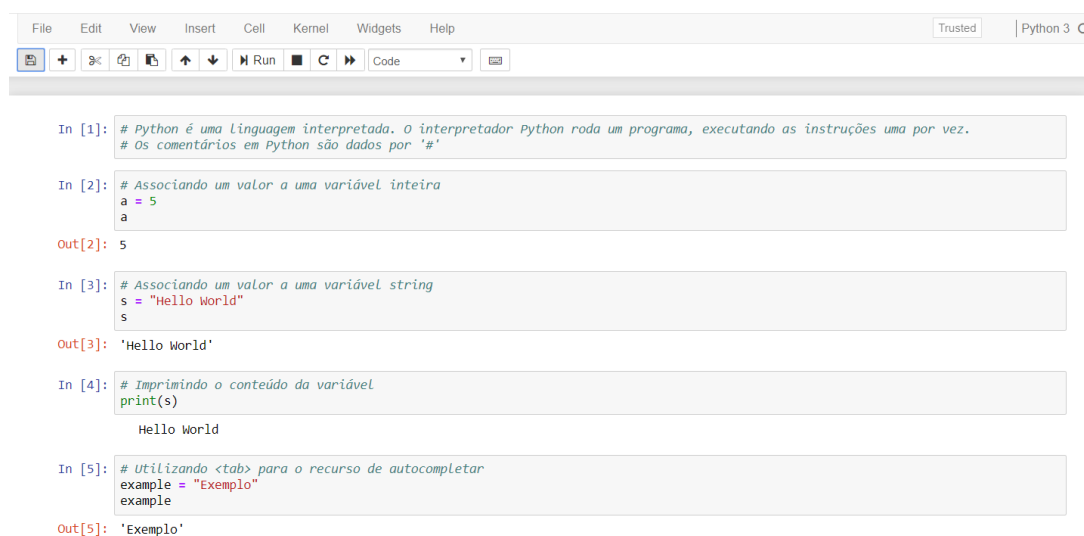
² Disponível em: <https://colab.research.google.com/notebooks/intro.ipynb?utm_source=scs-index>. Acesso em: 27 ago. 2021.

Figura 13 – Ambiente do Jupyter Notebook, mostrando uma pasta específica com arquivos de scripts



O Jupyter Notebook implementa uma maneira diferenciada de execução de scripts. Cada script pode ser colocado em uma **célula de entrada de dados**, com o resultado da sua execução em uma célula de saída, como a apresentada a seguir (Figura 14). O arquivo com diversos scripts é salvo no formato *ipynb*, podendo ser recuperado mais tarde com a mesma configuração da última execução efetuada.

Figura 14 – Exemplo de script em Jupyter Notebook: células de entrada de dados e execução

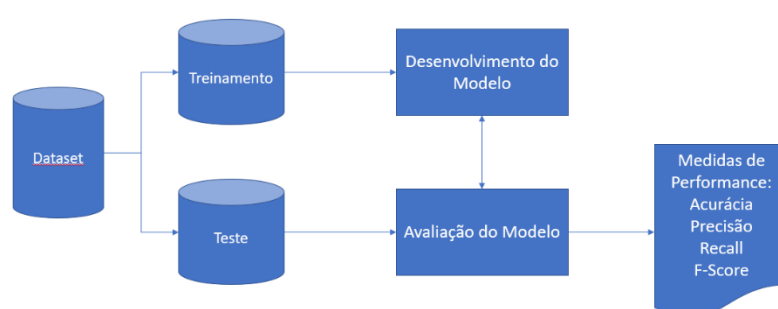


TEMA 4 – CONSTRUÇÃO DO MODELO EM PYTHON E JUPYTER NOTEBOOK

Após uma breve exposição da ferramenta Jupyter Notebook, a proposta é construir um MLP para exemplificar a aprendizagem de um processo de classificação. É comum utilizar *datasets* existentes em repositórios para testar

modelos de redes neurais. Um dos repositórios mais populares é o UCI Machine Learning Repository³. Apresenta em torno de 588 *datasets*, com 442 específicos para tarefas de classificação. Para a experimentação de um modelo de MLP, um *dataset* é escolhido, sendo dividido em dois conjuntos, o de treinamento e o de teste. O de treinamento será utilizado para o desenvolvimento do modelo, enquanto o de teste será utilizado somente para a avaliação. No processo de avaliação, serão adotadas algumas medidas de performance, conforme ilustra o diagrama da figura.

Figura 15 – Diagrama para construção do modelo de MLP



O *dataset* escolhido será o Wine Quality⁴ (Cortez et al., 2009). O *dataset* refere-se à classificação de qualidade de vinhos a partir de suas características químicas, como acidez volátil, alcalinidade, presença de substâncias químicas e ainda outras características, como a cor. No total, são 1599 instâncias (amostras) de vinhos, contendo 12 atributos ou características (considerando somente o *dataset* de vinhos tintos). Um recorte deste *dataset* pode ser visualizado na figura.

Figura 16 – *Dataset* Wine Quality: características (colunas), rótulo (classe) e instâncias (amostras, nas linhas)

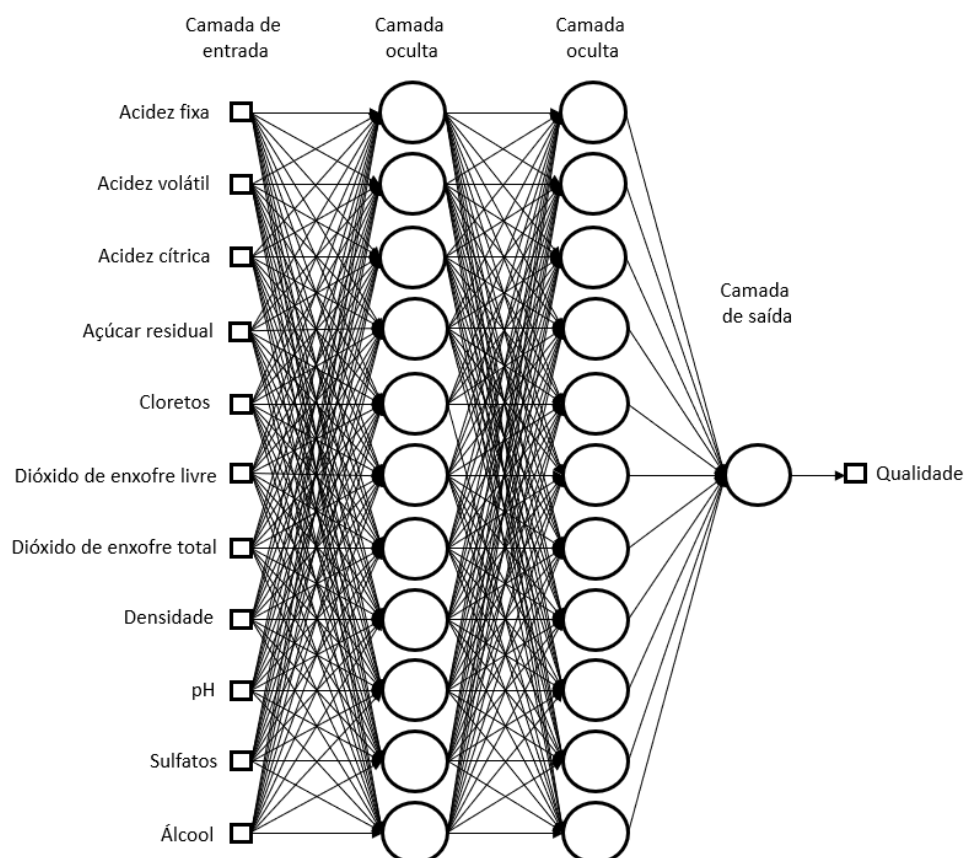
Características:												Rótulo:
	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
...												...

³ Disponível em: <<https://archive.ics.uci.edu/ml/index.php>>. Acesso em: 27 ago. 2021.

⁴ Disponível em: <<https://archive.ics.uci.edu/ml/datasets/Wine+Quality>>. Acesso em: 27 ago. 2021.

A construção de um MLP para a classificação de vinhos, conforme o *dataset* Wine Quality, requer que as entradas da rede sejam as características, tendo uma saída relativa à classe ou rótulo a que pertence o vinho, que expressa a sua qualidade. Com o *dataset* já classificado previamente, utiliza-se uma parte do conjunto total de instâncias para compor o conjunto de treinamento, e outra apenas para o teste da MLP gerada. Conforme a Figura 17, a arquitetura do MLP contará com um conjunto de 11 entradas (relativas a cada característica das amostras), 1 saída (relativa à qualidade do vinho), e 2 camadas ocultas, cada uma com 11 neurônios.

Figura 17 – Modelo MLP: problema de classificação de qualidade de vinho



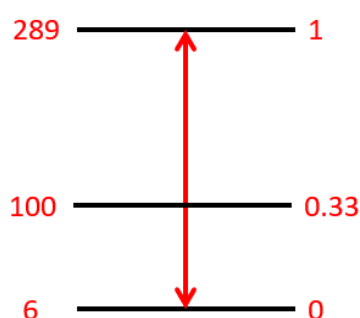
Podemos notar que as características apresentam faixas de valores bem distintas. Por exemplo, consultando diretamente no *dataset*⁵, os valores da acidez volátil (*volatile acidity*) estão na faixa entre 0,12 e 1,58. Já o valor de dióxido de enxofre total (*total sulfur dioxide - TSD*) aparece na faixa de 6 a 289. Esse aspecto pode ser um problema para a rede, pois os pesos sinápticos deverão assumir

⁵ Isso é feito por meio do comando *describe*, que faz parte do tipo *DataFrame* da biblioteca *Pandas*. Esse comando gera a estatística descritiva dos dados.

valores que estarão, de certa forma, desbalanceados quando comparados entre si. Para evitar essa situação, insere-se no modelo de MLP uma fase chamada de **normalização**, em que a faixa de valores diversos tende a ser padronizada para uma única faixa de valores.

Podemos normalizar os valores para uma faixa específica, como $[0,1]$. Ou seja, os neurônios assumem valores de entrada apenas dentro dessa faixa. É necessário, então, transformar os valores de entradas, para que fiquem dentro dessa faixa. A Figura 18 ilustra como a variável TSD pode ser normalizada. O valor mínimo que a variável pode assumir é 6; então, o neurônio transformará o valor para “0”. O valor máximo que ele poderá assumir é de 289; então, esse valor corresponderá ao valor do neurônio “1”. A normalização pode ser obtida por meio de regra de três simples. No caso de um valor qualquer, digamos, 100, o cálculo mostra que, na proporção da faixa normalizada $[0,1]$, o valor correspondente será de 0,33.

Figura 18 – Exemplo de normalização da característica TSD (dióxido de enxofre total): faixa $[0,1]$



Para a avaliação do modelo, utilizamos o conceito da **matriz de confusão**. A matriz de confusão é uma ferramenta muito interessante para o cálculo de indicadores que permitirão a medida de performance da classificação de um MLP ou ainda de outros tipos de RNA. Também conhecida como **matriz de contingência** ou **matriz de erro**, refere-se ao cruzamento entre as classes preditas pelo modelo construído contra os valores dados pela classe original. Um modelo acerta quando estima um valor positivo, que de fato deveria ser positivo, ou quando estima um valor negativo quando se espera negativo (Figura 19). No entanto, ele erra quando um valor deveria ser positivo, quando na verdade é negativo (erro tipo I), ou quando prediz como negativo o que na verdade é positivo

(erro tipo II). A matriz de confusão mostrada na figura ilustra as diferentes condições para o caso de uma avaliação binária. Esse tipo de avaliação pode ser estendida para a existência de mais classes.

Figura 19 – Interpretação da matriz de confusão

		Classe Original	
		Condição Positiva	Condição Negativa
Predição de Classe	Predição Positiva	Verdadeiro Positivo	Falso Negativo (Erro Tipo I)
	Predição Negativa	Falso Positivo (Erro Tipo II)	Verdadeiro Negativo

A classificação binária indica, portanto, quatro situações que a matriz de confusão pode assumir:

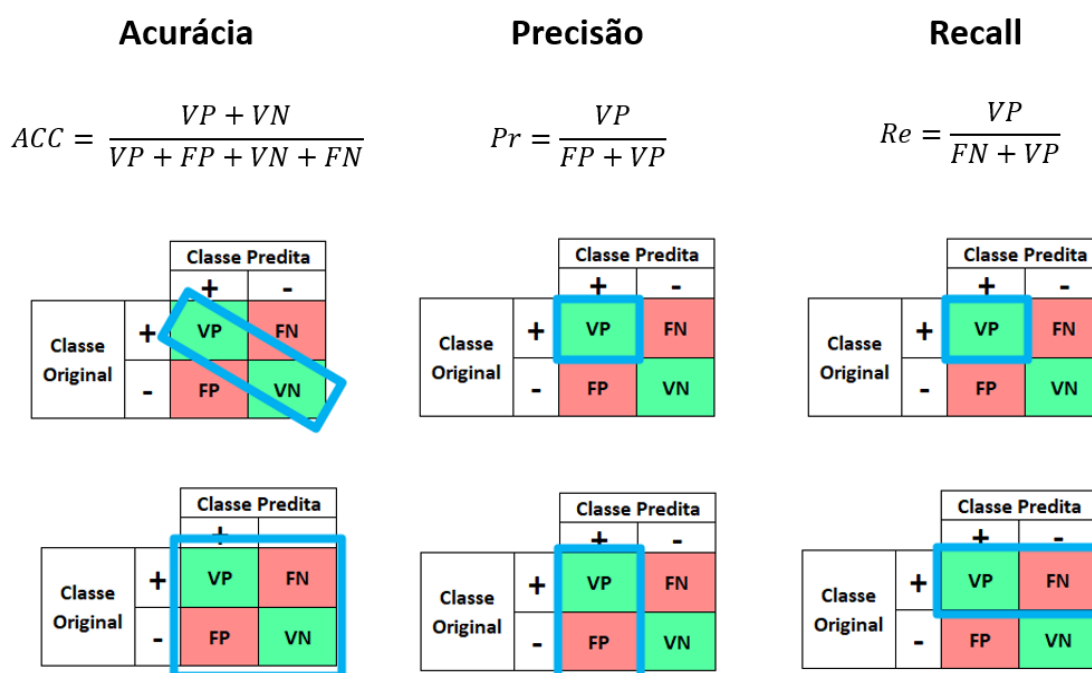
- **VP (Verdadeiro Positivo):** objeto da classe **positiva** classificado como **positivo** (por exemplo, um e-mail spam é classificado como spam).
- **VN (Verdadeiro Negativo):** objeto da classe **negativa** classificado como **negativo** (por exemplo, um e-mail normal é classificado como normal).
- **FP (Falso Positivo):** objeto da classe **negativa** classificado como **positivo** (por exemplo, e-mail normal é classificado como spam). Também chamado de *alarme falso* ou *erro tipo I*.
- **FN (Falso Negativo):** objeto da classe **positiva** classificado como **negativo** (por exemplo, e-mail spam é classificado como normal). Conhecido também como *erro tipo II*.

Da matriz de confusão, obtemos três indicadores: **acurácia**, **precisão** e **recall**. Na Figura 20, vemos as fórmulas para o cálculo de cada indicador.

- **Acurácia:** taxa global de sucesso do modelo. É o número de classificações corretas dividido pelo número total das classificações. A acurácia também pode ser entendida como o grau de conformidade de um valor predito com o seu valor real.

- **Precisão:** é a probabilidade de que um item recuperado seja relevante. Calcula-se o valor de verdadeiros positivos dividido pelo total de verdadeiros e falsos positivos.
- **Recall:** é a probabilidade de um item relevante ser recuperado. É o valor de verdadeiros positivos dividido pelo total dos verdadeiros positivos e falsos negativos. Se deveria resultar apresentar verdadeiro positivo, e é pelo menos falso negativo, então é um resultado relevante.

Figura 20 – Cálculo dos indicadores com base na matriz de confusão



Outro indicador geralmente utilizado é o **F-score**, que considera a combinação entre a precisão e o *recall*, calculado a partir da seguinte fórmula:

$$F = \frac{2 \cdot Pr \cdot Re}{Pr + Re}$$

Tais indicadores permitem a análise da performance da rede MLP (bem como de outras redes), viabilizando a validação do conjunto de teste e, consequentemente, da qualidade de generalização da rede MLP.

TEMA 5 – EXPERIMENTAÇÃO DO MODELO

Neste tema, vamos mostrar, no ambiente do Jupyter Notebook, a implementação do MLP, cujos detalhes da modelagem foram abordados

anteriormente. Vamos mostrar o conteúdo de cada célula, de forma a apresentar uma estrutura geral sobre como outros problemas de classificação podem ser adaptados para o caso do problema de qualidade de vinhos. Apesar de escrito em linguagem Python, os scripts são intuitivos, pois demonstram cada passo considerado na construção da rede MLP. Os scripts fazem parte de um arquivo único do Jupyter Notebook.

Em primeiro lugar, é necessário fazer a importação das bibliotecas que vão ser utilizadas. Na Figura 21, apresentamos o conjunto de bibliotecas em uma única célula. A execução pode levar algum tempo, pois é feita de maneira interna, não transparente ao usuário. Notamos as bibliotecas Numpy, Pandas, Matplotlib e vários módulos da biblioteca Sklearn, que contêm as implementações necessárias para trabalhar com redes neurais, bem como o MLP.

Figura 21 – Importação das bibliotecas

```
In [1]: import numpy as np
import pandas as pd
from matplotlib import pyplot as plt
import warnings
import sklearn.exceptions
from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import classification_report, confusion_matrix
```

O passo seguinte é a importação dos dados do problema de classificação de vinhos, processo que pode ser feito de duas formas: i) download do arquivo e instalação em pasta local; ou ainda ii) apontando diretamente para a pasta na URL do repositório UCI. A célula na Figura 22 mostra o segundo caso. O arquivo com os dados está no formato CSV, e assim utilizamos o comando *read_csv()* da biblioteca Pandas. Os dados são colocados em uma estrutura *DataFrame*. Logo em seguida, é solicitada a impressão do cabeçalho e das primeiras linhas do dataset com o comando *head()* do Pandas.

Figura 22 – Execução do comando de importação do arquivo do dataset (formato CSV)

```
In [2]: df = pd.read_csv('http://archive.ics.uci.edu/ml/machine-learning-databases/wine-quality/winequality-red.csv', sep=';')
df.head()

Out[2]:
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

O próximo comando (Figura 23) é executado para evidenciar a estatística descritiva dos dados, por meio do comando *describe()* do Pandas. Ele não é essencial para a execução do MLP, apenas para mostrar ao usuário a estrutura estatística dos dados. No processo de modelagem, esse passo pode ser interessante para mostrar se o dataset apresenta algum problema por conta de dados faltantes, o que exigiria uma etapa de pré-processamento dos dados. Podemos verificar o cálculo da contagem de instâncias, média aritmética, mediana (50%), desvio-padrão, valores mínimo e máximo, 1º quartil (25%) e 3º quartil (75%). O comando adicional *transpose()* simplesmente inverte linhas para colunas e vice-versa.

Figura 23 – Estatística descritiva dos dados

```
In [3]: df.describe().transpose()

Out[3]:
```

	count	mean	std	min	25%	50%	75%	max
fixed acidity	1599.0	8.319637	1.741096	4.60000	7.1000	7.90000	9.200000	15.90000
volatile acidity	1599.0	0.527821	0.179060	0.12000	0.3900	0.52000	0.640000	1.58000
citric acid	1599.0	0.270976	0.194801	0.00000	0.0900	0.26000	0.420000	1.00000
residual sugar	1599.0	2.538806	1.409928	0.90000	1.9000	2.20000	2.600000	15.50000
chlorides	1599.0	0.087467	0.047065	0.01200	0.0700	0.07900	0.090000	0.61100
free sulfur dioxide	1599.0	15.874922	10.460157	1.00000	7.0000	14.00000	21.000000	72.00000
total sulfur dioxide	1599.0	46.467792	32.895324	6.00000	22.0000	38.00000	62.000000	289.00000
density	1599.0	0.996747	0.001887	0.99007	0.9956	0.99675	0.997835	1.00369
pH	1599.0	3.311113	0.154386	2.74000	3.2100	3.31000	3.400000	4.01000
sulphates	1599.0	0.658149	0.169507	0.33000	0.5500	0.62000	0.730000	2.00000
alcohol	1599.0	10.422983	1.065668	8.40000	9.5000	10.20000	11.100000	14.90000
quality	1599.0	5.636023	0.807569	3.00000	5.0000	6.00000	6.000000	8.00000

Em seguida, é necessário compor o conjunto de treinamento. Aqui, será excluído do conjunto o rótulo ou classe *qualidade*, permanecendo apenas os

atributos que servem de entrada para o MLP. O comando *drop()* do Pandas retira do DataFrame a coluna com o rótulo *quality*. O conjunto de amostras do MLP ficará na variável do tipo DataFrame **X**.

Figura 24 – Extração da coluna de rótulo ou classe *quality* para composição do conjunto de amostras do MLP

```
In [4]: X = df.drop('quality', axis=1)
X
```

```
Out[4]:
```

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol
0	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	0.56	9.4
1	7.8	0.880	0.00	2.6	0.098	25.0	67.0	0.99680	3.20	0.68	9.8
2	7.8	0.760	0.04	2.3	0.092	15.0	54.0	0.99700	3.26	0.65	9.8
3	11.2	0.280	0.56	1.9	0.075	17.0	60.0	0.99800	3.16	0.58	9.8
4	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	0.56	9.4
...	...	...	...	...	...	...	...	...	...	...	...
1594	6.2	0.600	0.08	2.0	0.090	32.0	44.0	0.99490	3.45	0.58	10.5
1595	5.9	0.550	0.10	2.2	0.062	39.0	51.0	0.99512	3.52	0.76	11.2
1596	6.3	0.510	0.13	2.3	0.076	29.0	40.0	0.99574	3.42	0.75	11.0
1597	5.9	0.645	0.12	2.0	0.075	32.0	44.0	0.99547	3.57	0.71	10.2
1598	6.0	0.310	0.47	3.6	0.067	18.0	42.0	0.99549	3.39	0.66	11.0

1599 rows × 11 columns

Na Figura 25, a coluna do rótulo será colocada no vetor **y**. O comando apenas atribui a coluna do DataFrame.

Figura 25 – Atribuição dos rótulos para o vetor **y**

```
In [5]: y = df['quality']
y
```

```
Out[5]: 0      5
        1      5
        2      5
        3      6
        4      5
        ..
        1594    5
        1595    6
        1596    6
        1597    5
        1598    6
        Name: quality, Length: 1599, dtype: int64
```

O próximo comando fará a divisão do dataset nos conjuntos de treinamento e de teste (Figura 26). O comando *train_test_split()* faz essa tarefa, com o parâmetro *test_size*, que apresenta o tamanho que terá o conjunto de teste. No caso, 1/3 das instâncias ficarão com o conjunto de teste e 2/3 com o conjunto de

treinamento. A escolha é feita de forma aleatória (recordando do conceito de validação cruzada, visto anteriormente). Os *arrays* **X_train**, **X_test**, **y_train** e **Y_test** conterão dos dados necessários para as fases de treinamento e de teste do MLP.

Figura 26 – Divisão do conjunto das amostras em conjunto de treinamento e de teste

```
In [6]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.333)
```

```
In [7]: X_train, X_test
```

```
Out[7]: (   fixed acidity  volatile acidity  citric acid  residual sugar  chlorides  \
487          10.2           0.645         0.36         1.8         0.053
125           9.0           0.620         0.04         1.9         0.146
455          11.3           0.620         0.67         5.2         0.086
45           4.6           0.520         0.15         2.1         0.054
650          10.7           0.430         0.39         2.2         0.106
...          ...           ...           ...           ...           ...
273           8.4           0.715         0.20         2.4         0.076
864           7.2           0.620         0.06         2.7         0.077
1258          6.8           0.640         0.00         2.7         0.123
62            7.5           0.520         0.16         1.9         0.085
1315          7.5           0.630         0.27         2.0         0.083
```

A próxima etapa é a normalização dos dados. O objeto *StandardScaler()* é criado e o comando *fit()* do objeto é executado, de forma a criar a estrutura necessária para a normalização dos dados. Em seguida, **X_train** e **X_test** serão repostos com os dados normalizados a partir da chamada ao comando *transform()*. Essa etapa é visualizada na Figura 27.

Figura 27 – Normalização do conjunto de treinamento e de teste utilizando *StandardScaler()*

```
In [8]: scaler = StandardScaler()
scaler.fit(X_train)
```

```
Out[8]: StandardScaler()
```

```
In [9]: X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
X_train, X_test
```

```
Out[9]: (array([[ 1.1622524 ,  0.64077297,  0.48210577, ..., -0.97738772,
-1.39357493, -0.38246151],
[ 0.44105266,  0.50032308, -1.17655202, ..., -1.04467188,
0.26619222, -0.96163218],
[ 1.82335216,  0.50032308,  2.0889305 , ..., -0.64096689,
0.20691482,  2.89950558],
...,
```

Depois de todas essas etapas, procede-se ao treinamento do MLP propriamente dito. Utiliza-se o objeto *MLPClassifier()* para criar a rede, passando todos os parâmetros necessários à sua construção. No Quadro 2, apresentamos os principais parâmetros para a configuração do classificador MLP. É possível identificar o número de camadas ocultas (e a quantidade de neurônios em cada camada), a forma de atualização da taxa de aprendizagem, o número máximo de iterações, a tolerância do erro global (loss) para a parada do algoritmo de treinamento, a função de ativação, o algoritmo para treinamento e as taxas iniciais de aprendizagem e de momento.

Quadro 2 – Principais parâmetros para a configuração do MLP

Parâmetro	Descrição	Valores
hidden_layer_sizes	Definição de camadas ocultas, com o número de neurônios em cada uma (número ou tupla)	hidden_layer_sizes=4 hidden_layer_sizes=(4,3)
learning_rate	Tipo de taxa de treinamento ou aprendizagem ('constant', 'invscaling', 'adaptive')	learning_rate='adaptive'
max_iter	Número máximo de iterações do algoritmo.	max_iter=1000
tol	Tolerância para o erro global (loss). Caso esteja abaixo, o algoritmo irá parar.	tol=0.0001 tol=1e-4
activation	Função de ativação utilizada internamente pelo algoritmo. ('logistic', 'tanh', 'relu', 'identity')	activation='logistic'
solver	Algoritmo utilizado para o treinamento. ('sgd', 'lbfgs', 'adam')	solver='sgd'
learning_rate_init	Valor inicial da taxa de treinamento (entre 0 e 1)	learning_rate_init=0.9
momentum	Valor de inércia do treinamento ("o quanto o treinamento não deve mudar")	momentum=0.1

A Figura 28 apresenta os parâmetros utilizados no objeto *MLPClassifier()* para o problema de classificação de qualidade de vinhos. Como já modelado, a arquitetura do MLP conta com duas camadas ocultas de 11 neurônios cada. A função de ativação é a função logística já estudada anteriormente. O número máximo de iterações é 10000. O algoritmo utilizado para o treinamento é o de descida de gradiente estocástico ('sgd'), conforme o algoritmo de retropropagação que já estudamos. A taxa de aprendizagem é adaptativa, no sentido de que ela se modifica ao longo do treinamento – inicialmente, começa com 0,8, apresentando taxa de momento de 0,3.

Em seguida, o comando *fit()* do objeto *MLPClassifier* criado é executado. Note que são informados como parâmetros o conjunto de treinamento **X_train** e os rótulos **y_train**. As iterações do treinamento são visualizadas logo a seguir,

mostrando o status do erro global (*loss*). Cada execução do algoritmo acontece de forma única, devido à característica estocástica do algoritmo de treinamento. No caso da Figura 28, o erro ficou em aproximadamente 0,78264. A cada iteração, não há diminuição, indicando que o algoritmo chegou no mínimo global após 10 épocas (iterações) consecutivas com o mesmo valor de erro, finalizando na iteração de número 3044.

Figura 28 – Execução do treinamento do MLP

```
In [10]: mlp = MLPClassifier(hidden_layer_sizes=(11,11), activation='logistic', max_iter=10000, alpha=1e-6,
                             solver='sgd', verbose=10, tol=1e-9, random_state=1, learning_rate='adaptive',
                             momentum=.3, learning_rate_init=.8)

In [11]: mlp.fit(X_train,y_train)
Iteration 3031, loss = 0.78264006
Iteration 3032, loss = 0.78264006
Iteration 3033, loss = 0.78264006
Iteration 3034, loss = 0.78264006
Iteration 3035, loss = 0.78264006
Iteration 3036, loss = 0.78264006
Iteration 3037, loss = 0.78264006
Iteration 3038, loss = 0.78264006
Iteration 3039, loss = 0.78264006
Iteration 3040, loss = 0.78264006
Iteration 3041, loss = 0.78264006
Iteration 3042, loss = 0.78264006
Iteration 3043, loss = 0.78264006
Iteration 3044, loss = 0.78264006
Training loss did not improve more than tol=0.000000 for 10 consecutive epochs. Learning rate too small. Stopping.

Out[11]: MLPClassifier(activation='logistic', alpha=1e-06, hidden_layer_sizes=(11, 11),
                        learning_rate='adaptive', learning_rate_init=0.8, max_iter=10000,
                        momentum=0.3, random_state=1, solver='sgd', tol=1e-09,
                        verbose=10)
```

Após o treinamento do MLP, executa-se o teste do modelo com o conjunto **X_test**. O comando *predict()* do objeto *MLPClassifier()* é chamado. São colocadas no array *predictions* as predições dadas pelo modelo treinado. Note que *predictions* é diferente dos dados previamente classificados. O MLP irá classificar o conjunto de teste de acordo com o treinamento efetuado. Na Figura 29, vemos o comando e o conteúdo do array com a qualidade predita pelo modelo para cada instância do conjunto de teste.

Figura 29 – Predição do conjunto de teste pelo modelo MLP treinado

```
In [12]: predictions = mlp.predict(X_test)
         predictions

Out[12]: array([7, 7, 6, 6, 5, 7, 6, 5, 6, 6, 7, 6, 6, 6, 5, 5, 7, 6, 6, 6, 5, 5,
                7, 5, 5, 5, 6, 5, 7, 4, 6, 7, 5, 5, 6, 5, 5, 5, 5, 5, 6, 5, 6,
                5, 6, 5, 6, 5, 5, 5, 6, 6, 5, 5, 6, 5, 5, 6, 6, 5, 5, 5, 6, 5, 6,
                6, 6, 6, 6, 6, 5, 5, 6, 5, 6, 6, 4, 6, 6, 5, 6, 5, 5, 5, 6, 5, 6,
                5, 6, 5, 7, 5, 5, 7, 5, 7, 6, 6, 5, 6, 6, 5, 6, 7, 5, 6, 5, 5, 6,
                7, 6, 5, 6, 6, 5, 6, 5, 5, 5, 6, 6, 5, 6, 5, 6, 5, 5, 6, 5, 6, 6,
                6, 5, 5, 5, 5, 5, 6, 6, 6, 5, 5, 6, 5, 5, 5, 7, 5, 5, 5, 5, 6, 5,
                6, 7, 5, 6, 6, 5, 6, 6, 6, 6, 7, 6, 7, 6, 5, 6, 5, 5, 5, 6, 5, 6,
                6, 6, 5, 6, 5, 5, 5, 7, 5, 5, 7, 5, 5, 6, 5, 5, 6, 5, 5, 5, 5, 5,
                6, 6, 6, 6, 6, 5, 6, 5, 6, 6, 5, 5, 5, 6, 6, 5, 6, 5, 6, 7, 5, 6,
                5, 6, 5, 5, 6, 6, 5, 5, 6, 6, 6, 6, 6, 6, 5, 6, 5, 6, 7, 6, 6,
                6, 5, 6, 5, 5, 5, 6, 5, 6, 5, 5, 6, 5, 5, 5, 5, 5, 6, 5, 6, 6, 5,
                6, 5, 6, 6, 5, 6, 6, 5, 4, 5, 6, 6, 6, 6, 5, 5, 5, 6, 5, 5, 5, 5,
                6, 5, 6, 5, 6, 5, 5, 5, 5, 5, 6, 5, 6, 6, 5, 6, 5, 6, 5, 7, 6, 6,
                5, 6, 6, 7, 5, 6, 5, 5, 6, 5, 6, 6, 6, 5, 5, 6, 5, 7, 5, 6, 6, 6,
                5, 5, 5, 5, 5, 6, 6, 5, 7, 5, 5, 5, 6, 6, 5, 5, 5, 6, 7, 7, 5, 7,
                6, 5, 5, 5, 5, 6, 6, 7, 6, 6, 6, 5, 6, 6, 7, 5, 6, 7, 5, 6, 5, 5,
                6, 5, 6, 6, 5, 5, 5, 5, 6, 6, 6, 5, 6, 5, 5, 5, 5, 6, 6, 7, 7, 5,
                7, 6, 6, 6, 6, 5, 7, 7, 6, 6, 6, 6, 5, 6, 5, 5, 7, 5, 7, 5, 5, 6,
                6, 6, 6, 5, 7, 6, 6, 6, 6, 5, 5, 5, 5, 5, 5, 6, 6, 5, 5, 7, 6, 6,
                5, 6, 5, 5, 6, 6, 5, 6, 5, 6, 6, 7, 6, 6, 6, 6, 5, 5, 6, 5, 6, 7,
                5, 6, 5, 5, 5, 5, 6, 6, 6, 5, 6, 7, 5, 6, 5, 5, 7, 5, 5, 6, 5, 6,
                6, 6, 7, 5, 7, 6, 6, 6, 6, 5, 6, 6, 6, 6, 7, 6, 6, 5, 6, 6, 6, 5,
                6, 6, 6, 4, 6, 7, 5, 7, 5, 5, 6, 5, 5, 5, 5, 6, 5, 5, 6, 4, 5,
                5, 6, 5, 7, 6], dtype=int64)
```

Com os dados da predição, utiliza-se a matriz de confusão para o cálculo dos indicadores de performance do MLP obtido. Perceba que a matriz de confusão ideal, sem erro de classificação, deveria mostrar todos os valores apenas na diagonal principal. No entanto, o MLP classifica com determinado nível de erro da classe predita em relação à classe prévia do conjunto de teste.

Figura 30 – Matriz de confusão gerada para a predição do MLP do problema de classificação de vinhos

```
In [13]: print(confusion_matrix(y_test, predictions))

[[ 0  2  2  0  0  0]
 [ 0  1  7  1  0  0]
 [ 0  2 174  60  3  0]
 [ 0  0  53 138 15  0]
 [ 0  0  5  36 30  0]
 [ 0  0  0  2  2  0]]
```

Na Figura 31, temos um relatório gerado com o comando `classification_report()`. Note que é calculado o indicador para cada classe (de 3 a 8), apresentando acurácia de 64%, média ponderada da precisão de 63%, recall

de 64% e f1-score de 63%. Tais indicadores mostram que o MLP poderia melhorar a sua performance de classificação. A partir desse relatório, podemos alterar os parâmetros do objeto *MLPClassifier()*, de maneira a obter resultados melhores de classificação.

Figura 31 – Relatório de indicadores do problema de classificação de vinhos

```
In [14]: warnings.filterwarnings("ignore", category=sklearn.exceptions.UndefinedMetricWarning)
print(classification_report(y_test, predictions))
```

	precision	recall	f1-score	support
3	0.00	0.00	0.00	4
4	0.20	0.11	0.14	9
5	0.72	0.73	0.73	239
6	0.58	0.67	0.62	206
7	0.60	0.42	0.50	71
8	0.00	0.00	0.00	4
accuracy			0.64	533
macro avg	0.35	0.32	0.33	533
weighted avg	0.63	0.64	0.63	533

Os indicadores mostram de forma geral o desempenho de classificação do MLP. É possível visualizar cada amostra e a sua classificação de maneira individual, a partir do script da Figura 32. Esse script é a diferença entre o conjunto predito e as classes que constavam previamente no *dataset*. Logo em seguida, todas aquelas instâncias discrepantes aparecem listadas, o que evidencia uma diferença em relação à classificação prévia. Entre parênteses, aparece o índice da instância; o segundo indica a discrepância da predição com a classificação prévia do conjunto de teste.

Figura 32 – Script de verificação individual: classificação das instâncias do problema

```
In [15]: e = y_test - predictions
for item in e.iteritems():
    if(item[1] != 0):
        print(item)
```

```
(378, -1)
(469, -1)
(1449, 1)
(447, -1)
(1476, -1)
(3, 1)
(855, 1)
(563, 1)
(899, -2)
(1429, -2)
(319, 1)
(1437, 1)
(613, -1)
```

A ideia do script em Jupyter Notebook é mostrar um procedimento padrão que pode ser utilizado com outros *datasets* para demonstrar o treinamento do MLP. Esse procedimento pode ser diferente de um problema para outro, mas em essência todos os passos necessários estão contemplados. Esse processo pode ser adotado também com outros tipos de RNA, diferentes do MLP.

REFERÊNCIAS

- CORTEZ, P. et al. Modeling wine preferences by data mining from physicochemical properties. **Decis. Support Syst.**, v. 47, p. 547–553, 2009.
- DUDA, R. O.; HART, P. E.; STORK, D. G. **Pattern classification**. 2. ed. New York: Wiley, 2001.
- FAHLMAN, S. E. Faster-Learning Variations on Back-Propagation: An Empirical Study. In: TOURETZKY, D. S.; HINTON, G. E.; SEJNOWSKI, T. J. (Org.) **Proceedings of the 1988 Connectionist Models Summer School**. San Francisco, CA: Morgan Kaufmann, 1989. p. 38–51.
- FLETCHER, R.; REEVES, C. M. Function minimization by conjugate gradients. **Comput. J.**, v. 7, p. 149–154, 1964.
- HAYKIN, S. **Redes neurais: princípios e prática**. Porto Alegre: Bookman, 2001.
- KALMAN, R. E. A New Approach to Linear Filtering and Prediction Problems. **Journal of Basic Engineering**, v. 82, n. 1, p. 35–45, 1960.
- KOPEC, D. **Classic Computer Science Problems in Python**. Shelter Island-NY: Manning Publications Co., 2019.
- MARQUARDT, D. W. An Algorithm for Least-Squares Estimation of Nonlinear Parameters. **Journal of the Society for Industrial and Applied Mathematics**, v. 11, n. 2, p. 431–441, 1963.
- MEDEIROS, L. F. de. **Inteligência artificial aplicada: uma abordagem introdutória**. Curitiba: InterSaberes, 2018.
- MÜLLER, A. C.; GUIDO, S. **Introduction to Machine Learning with Python and Scikit-Learn**. Newton (MA): O'Reilly, 2015.
- RIEDMILLER, M.; BRAUN, H. **RPROP: A Fast Adaptive Learning Algorithm**. 1992. Disponível em: <http://citeseerx.ist.psu.edu/viewdoc/summary?doi=10.1.1.52.4576>. Acesso em: 3 ago. 2021.